

► Números aleatorios

Bibliografía

- *Computational Physics*, Mark Newman.
- *Programming for computations - Python*, Evein Linge and Petter Langtangen.
- *Computational Physics with Python*, Eric Ayars.
- *Computational mathematics: An introduction to Numerical Analysis and Scientific Computing with Python*, Dimitrios Mitsotakis.
- *Computational physics: problem solving with Python*, Rubin H. Landau, Manuel J. Páez and Cristian C. Bordeianu.

Generador de congruencia lineal

Si nos piden que digamos números al azar, comenzamos a nombrar los primeros que se nos vengan a la cabeza: tal vez los que más nos gusten, tal vez solo positivos, tal vez números primos... La pregunta es: ¿en realidad son al azar? Supongamos que mejor se los pedimos al computador. ¿Resolvería esto el problema? Recordemos que el computador solo conoce ceros y unos: opera de forma estrictamente determinista y, en rigor, no puede generar azar verdadero.

*Decimos que una secuencia de números $x_0, x_1, \dots, x_n$ es aleatoria si no existe una correlación estadística entre ellos. A lo máximo que podemos aspirar con arquitecturas deterministas es a la generación de números **pseudoaleatorios**.*

El método clásico por excelencia es el **Generador de Congruencia Lineal (GCL)**: fijamos una semilla x_0 y, a partir de ella, generamos los términos sucesivos mediante la relación de recurrencia:

$$x_{n+1} = (ax_n + c) \pmod{m}.$$

La elección adecuada de los parámetros a (multiplicador), c (incremento) y m (módulo) determina la calidad y el período del generador.

Ejemplo

Tomemos $a = 5$, $c = 3$ con módulo $m = 8$. A partir de la semilla $x_0 = 1$, el GCL genera sucesivamente:

$$0, 3, 2, 5, 4, 7, 6, 1, 0, 3, \dots$$

Observamos que recorre todos los residuos posibles módulo 8 antes de volver a la semilla inicial

$x_0 = 1$, completando un período de longitud $P = 8$.

En la siguiente tabla se muestran los valores utilizados usualmente para los parámetros a, c, m .

Tabla 1: Parámetros de generadores lineales clásicos frente a generadores modernos.

Sistema / Implementación	Tipo	a	c	m
IBM RANDU (System/360)	LCG	65 539	0	2^{31}
ANSI C (rand MSVC)	LCG	214 013	2 531 011	2^{32}
POSIX drand48	LCG	25 214 903 917	11	2^{48}
Numerical Recipes (32-bit)	LCG	1 664 525	1 013 904 223	2^{32}
Java Random	LCG	25 214 903 917	11	2^{48}
NumPy (default_rng)	PCG64	6 364 136 223 846 793 005	Dinámico (128-bit)	2^{128}

- **IBM RANDU (década de 1960):** Fue el generador estándar instalado en las computadoras centrales (mainframes) IBM System/360. Diseñado en una época donde los ciclos de CPU eran carísimos, sus parámetros se eligieron para que la multiplicación se pudiera calcular con operaciones binarias ultrarrápidas (*shifts* y sumas). Es recordado como el peor generador ampliamente utilizado en la historia de la computación científica.
- **ANSI C (rand):** Es la función de aleatoriedad básica que venía por defecto en los compiladores estándar del lenguaje C (como Microsoft Visual C++ o Turbo C). Es un LCG de 32 bits elemental y bastante pobre para física computacional.
- **POSIX drand48:** La rutina estándar de los sistemas operativos UNIX y distribuciones Linux para generar números en coma flotante de doble precisión. Al usar 48 bits en lugar de 32, ofreció una mejora notable para la época.
- **Numerical Recipes (Press et al.):** Es la famosa “biblia” del cómputo científico. Sus autores propusieron valores específicos para a, c, m calculados cuidadosamente para maximizar el período en 32 bits ($m = 2^{32}$) sin sufrir las fallas evidentes de RANDU.
- **Mersenne Twister (MT19937) / NumPy Legacy:** Durante casi 20 años fue el rey indiscutible de la computación científica y el motor por defecto en `np.random.RandomState`, Python estándar (`random`), MATLAB y R. No es un LCG; funciona con matrices lineales y registros de desplazamiento sobre campos finitos. Su período es astronómico ($2^{19937} - 1$, un primo de Mersenne), pero consume mucha memoria de estado (unos 2.5 kilobytes) y falla pruebas estadísticas modernas muy rigurosas.
- **PCG64 (`np.random.default_rng`):** Significa *Permuted Congruential Generator* de 64 bits. Es el estándar moderno de NumPy desde la versión 1.17. Es una idea brillante: por debajo utiliza un LCG gigantesco de 128 bits para el estado interno, pero **nunca entrega el número tal cual**. Aplica una función no lineal de mezcla y rotación de bits a la salida (*output permutation*). Esto destruye cualquier patrón reticular o alineación geométrica, siendo rapidísimo, consumiendo poquísima memoria y superando todas las baterías de pruebas estadísticas actuales.

A continuación se muestra la generación de números aleatorios construyendo el GLC a mano y usando el generador de NumPy

numeros_aleatorios.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def lcg(size, seed=23):
5     """Genera un arreglo de N número entre (0,1) usando  $x_{n+1} = ax_n + c \bmod m$ 
6     """
7     a = 1103515245
8     c = 12345
9     m = 2**31
10
11     numeros = np.empty(size)
12     x0 = seed
13     xn = x0
14
15     for i in range(size):
16         xn = (a*xn + c) % m
17         numeros[i] = xn/m
18     return numeros
19
20 if __name__ == "__main__":
21     N = 10000
22     semilla = 12
23
24     #datos a mano con Linear congruence
25     datos_lcg = lcg(N,semilla) * 1000
26
27     # datos con numpy
28     rng = np.random.default_rng(seed=semilla) # instanciamos semilla
29     datos_numpy = rng.random(N) * 1000
30
31     # 3. Scatter Plots
32     fig, axes = plt.subplots(1, 2, figsize=(11, 5))
33
34     # Scatter LCG (A mano)
35     axes[0].scatter(datos_lcg[:-1], datos_lcg[1:], s=1, color='crimson',
36     alpha=0.5)
37     axes[0].set_title("Correlación  $x_n$  vs  $x_{n+1}$  (A mano - LCG)",
38     fontweight='bold')
39     axes[0].set_xlabel(" $x_n$ ")
40     axes[0].set_ylabel(" $x_{n+1}$ ")
41
42     # Scatter NumPy
43     axes[1].scatter(datos_numpy[:-1], datos_numpy[1:], s=1, color='teal',
44     alpha=0.5)
45     axes[1].set_title("Correlación  $x_n$  vs  $x_{n+1}$  (NumPy - PCG64)",
46     fontweight='bold')
47     axes[1].set_xlabel(" $x_n$ ")
```

```

44 axes[1].set_ylabel("$x_{n+1}$")
45
46 plt.tight_layout()
47
48 plt.savefig("../Clases/Herramientas_Python/random.pdf", bbox_inches='
tight')
49 plt.show()

```

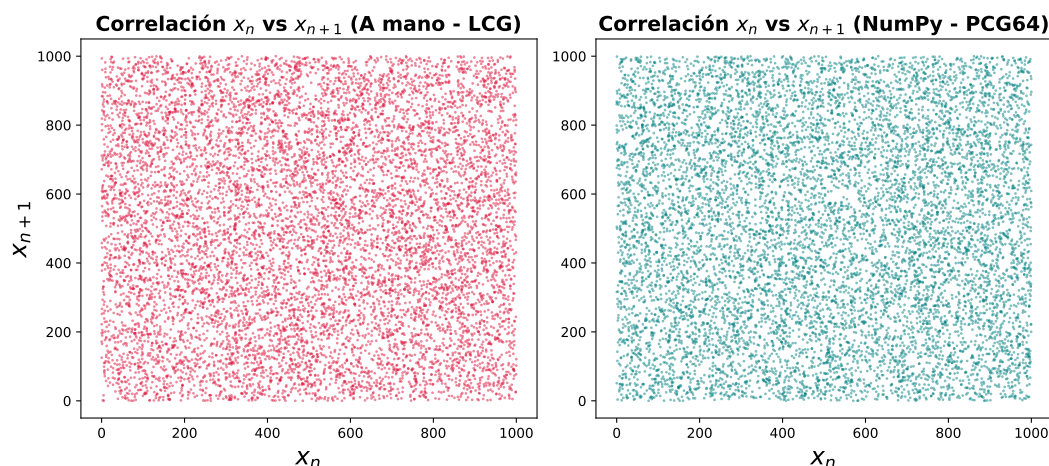


Figura 1: Generación de números aleatorios.

Desde la versión 1.17, NumPy separó por completo el problema de los números aleatorios en dos capas independientes:

1. **BitGenerators (El motor de bits en bruto):** Rutinas de bajo nivel escritas en C cuyo único trabajo es generar secuencias de enteros de 32 o 64 bits pseudoaleatorios a máxima velocidad.
2. **Generators (Las distribuciones estadísticas):** La interfaz de usuario creada con `default_rng()`, la cual toma los bits suministrados por el *BitGenerator* y los transforma analíticamente en variables continuas o discretas (Uniforme, Gaussiana, Poisson, Exponencial, etc.).

[Semilla (Seed)] $\longrightarrow$ BitGenerator (PCG64) $\xrightarrow{\text{bits crudos}}$ Generator (default_rng) $\longrightarrow$ Arreglos de datos

Figura 2: Esquema de dos capas del módulo `numpy.random`.

Aunque `np.random.default_rng()` utiliza **PCG64** por defecto, NumPy incluye cuatro motores intercambiables según el tipo de simulación:

- **PCG64 (Estándar recomendado):** Es un *Permuted Congruential Generator* de 64 bits. Internamente mantiene un estado de 128 bits gobernado por un LCG y un incremento impar variable. Sin embargo, no entrega los bits del LCG directamente: aplica una función de permutación no lineal sobre el estado para truncar y rotar los bits salientes. Combina excelente velocidad, poquísimos uso de memoria caché y aprueba las pruebas estadísticas más exigentes del estándar *BigCrush*.

- **PCG64DXSM (Recomendado para cómputo paralelo masivo):** Una variante de PCG que introduce una permutación multiplicativa más agresiva (*DXSM: Double-xorshift-multiply*). Se diseñó para evitar pequeños defectos de correlación que pueden surgir cuando se bifurcan millones de flujos aleatorios (*streams*) en clústeres paralelos o múltiples hilos.
- **MT19937 (Mersenne Twister):** El motor clásico basado en matrices de torsión sobre campos de Galois $\mathbb{F}_2$. Posee un período de $2^{19937} - 1$, pero consume un estado grande de 2.5 KB por instancia y es un 30–50 % más lento que PCG64. Se mantiene principalmente para reproducir código histórico o comparar simulaciones previas a 2019.
- **Philox y SFC64 (Criptográficos y computación GPU/HPC):**
 - **Philox** es un generador basado en cifrado por bloques (*counter-based*). Es ideal para GPUs y paralelismo masivo distribuido, ya que permite calcular el estado n -ésimo de la secuencia de forma directa en $\mathcal{O}(1)$ sin iterar los pasos previos.
 - **SFC64 (Small Fast Chaotic):** Basado en autómatas caóticos de 256 bits, es el generador más veloz en CPU, pasando también todas las pruebas estadísticas estándar.

Las distribuciones más utilizadas en física computacional

Tabla 2: Una vez instanciado el generador con `rng = np.random.default_rng(semilla)`, los métodos disponibles para generar datos son:

Método	Distribución / Operación	Aplicación física típica
<code>rng.random(size)</code>	Uniforme continua $U \in [0.0, 1.0)$	Algoritmos de Monte Carlo, ruletas
<code>rng.uniform(low, high, size)</code>	Uniforme continua en $[a, b)$	Posiciones iniciales homogéneas
<code>10**rng.uniform(a, b, size)</code>	Escala logarítmica continua $[10^a, 10^b]$	Barridos de parámetros (acoplamientos, masas)
<code>rng.lognormal(mean, sigma, size)</code>	Lognormal $\ln(X) \sim \mathcal{N}(\mu, \sigma)$	Tamaños de grano, turbulencia, astrofísica
<code>rng.logseries(p, size)</code>	Serie logarítmica discreta	Distribución de especies, percolación
<code>rng.integers(low, high, size)</code>	Enteros equiprobables en $[a, b)$	Espines en modelo de Ising, pasos discretos
<code>rng.normal(loc, scale, size)</code>	Gaussiana $\mathcal{N}(\mu, \sigma)$	Velocidades de Maxwell-Boltzmann, ruido térmico
<code>rng.exponential(scale, size)</code>	Exponencial $\lambda e^{-\lambda t}$	Decaimiento radiactivo, caminos libres medios
<code>rng.poisson(lam, size)</code>	Conteo de Poisson	Conteo de fotones en detectores, eventos raros
<code>rng.choice(a, size, replace, p)</code>	Muestreo discreto ponderado	Cadenas de Markov (Metropolis-Hastings)

Dato curioso curioso: el desastre de RANDU y el Teorema de Marsaglia

El generador de IBM, RANDU, utilizaba los parámetros:

$$x_{n+1} = 65539 x_n \pmod{2^{31}}, \quad \text{con } c = 0.$$

El valor $a = 65539$ no fue fruto del rigor estadístico, sino de la pereza del hardware: en base 2, $65539 = 2^{16} + 3$. Esto permitía a los programadores calcular la multiplicación $a \cdot x_n$ simplemente desplazando bits a la izquierda 16 posiciones ($x_n \ll 16$) y sumándole $x_n + 2x_n$, evitando la costosa instrucción de multiplicación en el procesador.

El costo científico fue catastrófico. En 1968, el matemático George Marsaglia publicó un célebre artículo titulado *Random Numbers Fall Mainly in the Planes* (Los números aleatorios caen principalmente en los planos), demostrando que cualquier generador congruencial lineal sufre de regularidades reticulares.

En el caso específico de RANDU, si tomamos tres números consecutivos x_n, x_{n+1}, x_{n+2} :

$$\begin{aligned} x_{n+2} &= (2^{16} + 3)x_{n+1} \pmod{2^{31}} \\ &= (2^{16} + 3)^2 x_n \pmod{2^{31}} \\ &= (2^{32} + 6 \cdot 2^{16} + 9)x_n \pmod{2^{31}}. \end{aligned}$$

Dado que $2^{32} \equiv 0 \pmod{2^{31}}$, y manipulando algebraicamente los términos con $x_{n+1} = (2^{16} + 3)x_n$, se deduce la relación lineal exacta:

$$x_{n+2} - 6x_{n+1} + 9x_n \equiv 0 \pmod{2^{31}}.$$

Si graficamos ternas de puntos consecutivos (x_n, x_{n+1}, x_{n+2}) en el espacio tridimensional $\mathbb{R}^3$, en lugar de llenar homogéneamente el cubo unitario como esperaríamos de un gas o un medio aleatorio, **todos los puntos se concentran exclusivamente sobre apenas 15 planos bidimensionales paralelos**. Décadas de simulaciones de Monte Carlo en física nuclear, termodinámica y astrofísica realizadas en los años 60 y 70 tuvieron que ser descartadas o recalculadas porque los resultados reflejaban la geometría de estos 15 planos y no la física del problema.

Ejemplo: Estimación del valor de π

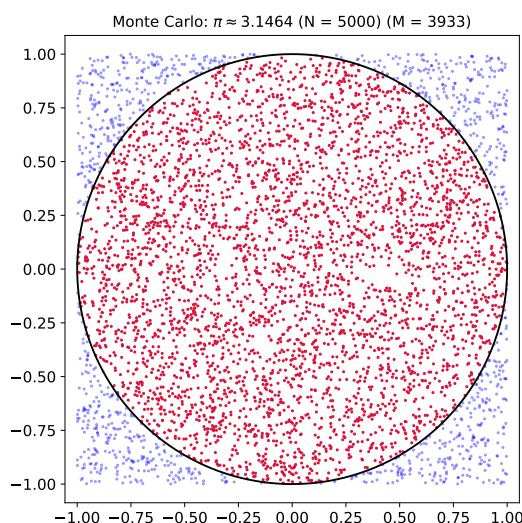


Figura 3: Estimación de π usando números aleatorios.

Una de las aplicaciones simples de los números aleatorios es el cálculo de π . Imagina que inscribimos

un círculo de radio R centrado en el origen en un cuadrado de lado $L = 2R$, delimitado por $[-R, R] \times [-R, R]$. Tomemos por comodidad $R = 1$ u. Sabemos que el área del cuadrado $A_{\square} = L \times L = (2R)^2 = 4 u^2$. Por otro lado, el área del círculo inscrito es: $A_{\circ} = \pi R^2 = \pi u^2$.

Suponga entonces que generamos N puntos aleatorios en rectángulo $[-1, 1] \times [-1, 1]$. De estos puntos, claramente no todos van a caer dentro del círculo. Digamos que unos M caen en el interior del círculo. Entonces, $\frac{A_{\circ}}{A_{\square}} = \frac{M}{N}$ es la probabilidad de que un punto aleatorio caiga dentro del círculo. Luego, $\pi = 4 \frac{M}{N}$. Es decir, solo tenemos que contar el número de puntos que caen en el interior del círculo y dividirlos por los totales, entonces si división es proporcional a π .

pi_random.py

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4
5 def monte_carlo_pi(N=5000, seed=42):
6     rng = np.random.default_rng(seed)
7     x = rng.uniform(-1, 1, N)
8     y = rng.uniform(-1, 1, N)
9     adentro = x**2 + y**2 <= 1.0 # se define el disco de radio 1
10    M = np.sum(adentro) # suma todos los puntos adentro del círculo
11    pi_ran = 4.0 * M / N
12    return x, y, adentro, pi_ran, M
13
14
15 if __name__ == "__main__":
16     N = 5000
17     x, y, adentro, pi_aprox, M = monte_carlo_pi(N)
18
19     plt.figure(figsize=(6, 6), facecolor='white')
20     plt.scatter(x[adentro], y[adentro], s=2, color="crimson")
21     plt.scatter(x[~adentro], y[~adentro], s=2, color="blue", alpha=0.4)
22
23     # Frontera
24     theta = np.linspace(0, 2 * np.pi, 200)
25     plt.plot(np.cos(theta), np.sin(theta), "k-", lw=1.5)
26
27     plt.axis("equal")
28     plt.xlim(-1.05, 1.05)
29     plt.ylim(-1.05, 1.05)
30     plt.title(f"Monte Carlo:  $\pi \approx \{pi\_aprox:.4f\}$  (N = {N}) (M = {M})")
31     plt.tick_params(axis='both', which='both', labelsize=12)
32     plt.tight_layout()
33     plt.savefig('../Clases/Herramientas_Python/pirandom.pdf', bbox_inches='tight')
34
35     plt.show()

```

Ejemplo 2: camino de una persona borracha en 1D

Imaginemos a una persona caminante sobre una línea recta unidimensional que parte del origen $x_0 = 0$ en el instante $t = 0$. En cada intervalo de tiempo discreto $\Delta t = 1$ s, esta persona da un paso de longitud fija $\ell = 1$ m:

- Hacia la derecha (+1) con probabilidad $1/2$.
- Hacia la izquierda (−1) con probabilidad $q = 1 - 1/2 = 1/2$.

Definimos el i -ésimo paso como una variable aleatoria discreta $s_i \in \{-1, +1\}$ tal que:

$$P(s_i = +1) = \frac{1}{2}, \quad P(s_i = -1) = \frac{1}{2}. \quad (1)$$

Las propiedades estadísticas de un paso individual son inmediatas:

$$\langle s_i \rangle = (+1) \left(\frac{1}{2} \right) + (-1) \left(\frac{1}{2} \right) = 0, \quad (2)$$

$$\langle s_i^2 \rangle = (+1)^2 \left(\frac{1}{2} \right) + (-1)^2 \left(\frac{1}{2} \right) = 1, \quad (3)$$

$$\text{Var}(s_i) = \langle s_i^2 \rangle - \langle s_i \rangle^2 = 1. \quad (4)$$

La posición de la persona después de dar N pasos sucesivos es la suma de los pasos individuales:

$$X_N = \sum_{i=1}^N s_i. \quad (5)$$

Valor esperado (Desplazamiento medio): Por linealidad del valor esperado:

$$\langle X_N \rangle = \left\langle \sum_{i=1}^N s_i \right\rangle = \sum_{i=1}^N \langle s_i \rangle = 0. \quad (6)$$

En promedio, el caminante nunca se aleja del punto de partida.

Desplazamiento Cuadrático Medio: Para cuantificar cuánto se dispersa típicamente del origen, calculamos $\langle X_N^2 \rangle$:

$$X_N^2 = \left(\sum_{i=1}^N s_i \right) \left(\sum_{j=1}^N s_j \right) = \sum_{i=1}^N s_i^2 + \sum_{i \neq j} s_i s_j. \quad (7)$$

Tomando el promedio estadístico (ensamble) y recordando que los pasos son estadísticamente independientes ($\langle s_i s_j \rangle = \langle s_i \rangle \langle s_j \rangle = 0$ para $i \neq j$):

$$\langle X_N^2 \rangle = \sum_{i=1}^N \langle s_i^2 \rangle + \sum_{i \neq j} \langle s_i s_j \rangle = \sum_{i=1}^N 1 + 0 = N. \quad (8)$$

La ley de difusión $\sqrt{N}$

La distancia típica que se recorre respecto al origen (la desviación estándar o raíz del desplazamiento cuadrático medio, RMS) escala como la raíz cuadrada del número de pasos:

$$x_{\text{rms}} = \sqrt{\langle X_N^2 \rangle} = \sqrt{N}. \quad (9)$$

Si el número de pasos es proporcional al tiempo ($N \propto t$), encontramos la rúbrica fundamental del transporte difusivo: la distancia típica crece como $\sqrt{t}$, en contraste con el transporte balístico determinista donde la distancia crece como t .

drunk_walk.py

```

1 import math
2 import random
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6
7 def drunk_1d(N, L=1.0):
8     """
9     Simula caminante aleatorio en 1D.
10    Paso  $\Delta x \sim [-L, L]$ .
11    """
12    x = 0.0 #centrado en 0
13    x_history = [x]
14
15    for _ in range(N): #note que no uso el contador i porque no lo necesito
16        x += (random.random() - 0.5) * 2.0 * L # paso aleatorio entre -1 y 1
17        x_history.append(x)
18
19    return x_history
20
21
22 def drunk_2d(N, L=1.0):
23     """
24     Simula caminante aleatorio en 2D.
25     Pasos independientes  $\Delta x, \Delta y \sim [-L, L]$ .
26     """
27    x, y = 0.0, 0.0
28    x_history, y_history = [x], [y]
29
30    for _ in range(N):
31        x += (random.random() - 0.5) * 2.0 * L
32        y += (random.random() - 0.5) * 2.0 * L
33        x_history.append(x)
34        y_history.append(y)
35
36    return x_history, y_history
37

```

```
38 def drunk_3d(N, L=1.0):
39     """
40     Simula caminantx aleatorio en 3D.
41     Pasos independientes  $\Delta x, \Delta y, \Delta z \sim [-L, L]$ .
42     """
43     x, y, z = 0.0, 0.0, 0.0
44     x_history, y_history, z_history = [x], [y], [z]
45
46     for _ in range(N):
47         x += (random.random() - 0.5) * 2.0 * L
48         y += (random.random() - 0.5) * 2.0 * L
49         z += (random.random() - 0.5) * 2.0 * L
50         x_history.append(x)
51         y_history.append(y)
52         z_history.append(z)
53
54     return x_history, y_history, z_history
55
56
57 if __name__ == "__main__":
58
59     # -----
60     # PARÁMETROS GENERALES
61     # -----
62     N = 1000 # Número total de pasos
63     L = 10.0
64     n_pasos = list(range(N + 1))
65     semilla = 42
66
67     random.seed(semilla)
68     np.random.seed(semilla)
69
70     # 1D
71     x_1d = drunk_1d(N, L=L)
72     x_final_1d = x_1d[-1]
73
74     # 2D
75     x_2d, y_2d = drunk_2d(N, L=L)
76     x_final_2d, y_final_2d = x_2d[-1], y_2d[-1]
77
78     # 3D
79     x_3d, y_3d, z_3d = drunk_3d(N, L=L)
80     x_final_3d, y_final_3d, z_final_3d = x_3d[-1], y_3d[-1], z_3d[-1]
81
82     # -----
83     # PLOTS
84     # -----
85     fig = plt.figure(figsize=(18, 5))
86
87     # --- Subplot 1: Random Walk 1D (x vs Tiempo/Pasos) ---
```

```

88     ax1 = fig.add_subplot(1, 3, 1)
89     ax1.plot(n_pasos, x_1d, color='royalblue', linewidth=1.5, label='
Trayectoria $x(N)$')
90     ax1.plot(0, x_1d[0], 'go', markersize=7, label='Inicio')
91     ax1.plot(N, x_final_1d, 'ro', markersize=7, label='Fin')
92
93     ax1.set_title('Random Walk 1D', fontweight='bold')
94     ax1.set_xlabel('Número de pasos ($N$)')
95     ax1.set_ylabel('Posición $x$')
96     ax1.grid(True, linestyle='--', alpha=0.5)
97     ax1.legend(loc='upper left', fontsize=9)
98
99     # --- Subplot 2: Random Walk 2D (Plano X-Y) ---
100    ax2 = fig.add_subplot(1, 3, 2)
101    ax2.plot(x_2d, y_2d, color='royalblue', linewidth=1, alpha=0.85, label='
Trayectoria')
102    ax2.plot(0, 0, 'go', markersize=8, label='Inicio (0,0)')
103    ax2.plot(x_final_2d, y_final_2d, 'ro', markersize=8, label='Fin')
104
105    ax2.set_title('Random Walk 2D', fontweight='bold')
106    ax2.set_xlabel('Posición $x$')
107    ax2.set_ylabel('Posición $y$')
108    ax2.set_aspect('equal', 'datalim')
109    ax2.grid(True, linestyle='--', alpha=0.5)
110    ax2.legend(loc='upper right', fontsize=9)
111
112    # --- Subplot 3D: Espacio X-Y-Z ---
113    ax3 = fig.add_subplot(1, 3, 3, projection='3d')
114    ax3.plot(x_3d, y_3d, z_3d, color='royalblue', linewidth=1, alpha=0.85,
label='Trayectoria')
115    ax3.plot([0], [0], [0], 'go', markersize=7, label='Inicio (0,0,0)')
116    ax3.plot([x_3d[-1]], [y_3d[-1]], [z_3d[-1]], 'ro', markersize=7, label='
Fin')
117
118    ax3.set_title('Random Walk 3D', fontweight='bold')
119    ax3.set_xlabel('Posición $x$')
120    ax3.set_ylabel('Posición $y$')
121    ax3.set_zlabel('Posición $z$')
122    ax3.legend(loc='upper right', fontsize=8)
123
124    fig.tight_layout()
125    plt.savefig('../Clases/Herramientas_Python/drunk.pdf', bbox_inches='
tight')
126    plt.show()

```

En el código `animation_drunk.py` puede encontrar la animación para dos y tres dimensiones.

`julia_mandelbroot.py`

```

1 import matplotlib.pyplot as plt
2 import numpy as np

```

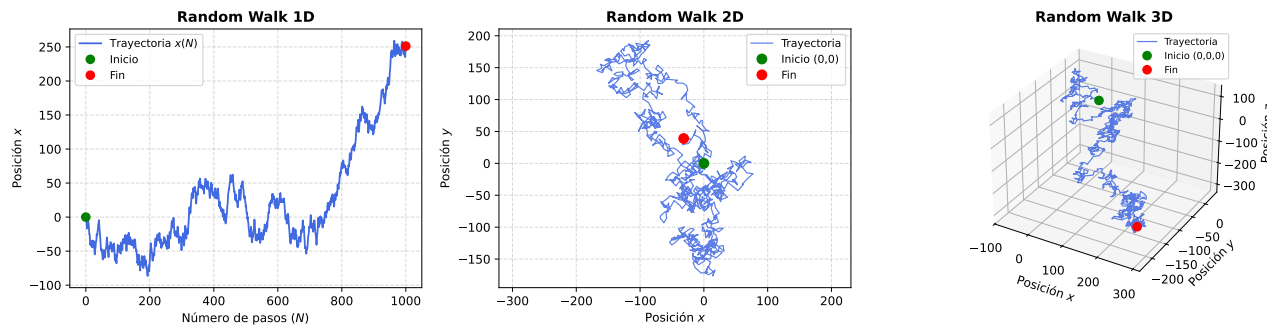


Figura 4: Random walk.

```

3
4
5 def mandelbrot(xmin=-2.0, xmax=0.5, ymin=-1.25, ymax=1.25, width=1000,
6     height=1000, max_iter=200):
7     """Calcula la matriz para el Conjunto de Mandelbrot.
8     Mantiene z0 = 0 y varía c.
9
10    Parámetros:
11
12    xmin, xmax: rango para el eje horizontal. parte real de c
13    ymin, ymax = rango para el eje vertical. parte imaginaria de c
14    width, height: resolución de la imagen (por defecto $1000 \times 1000 =
15    10^6$ puntos).
16    max_iter: El límite de iteraciones por punto. Si después de 200 pasos  $|z| < 2$ 
17    se asume que
18    pertenece al conjunto de Mandelbrot.
19
20    Devuelve:
21
22    escape_grid: Una matriz 2D de enteros de tamaño (height, width). Cada
23    celda (i, j)
24    almacena el número de iteraciones en las que ese punto escapó
25    (un valor entre $0$ y $199$), o $200$ (max_iter) si el punto nunca escapó
26    y
27    se quedó atrapado dentro del fractal.
28
29    (xmin, xmax, ymin, ymax): Una tupla con las coordenadas extremas.
30    Estas se usan para graficar con plt.imshow y graficar los valores físicos
31    en lugar del
32    índice de píxeles del arreglo.
33    """
34
35    x = np.linspace(xmin, xmax, width)
36    y = np.linspace(ymin, ymax, height)
37    C = x + 1j * y[:, None] # Matriz de parámetros c La coma crea una segunda
38    dimensión en el array
39    Z = np.zeros_like(C, dtype=np.complex128) # se crea la matriz de ceros

```

```
con la dimensión de C
33 escape_grid = np.full(C.shape, max_iter, dtype=int) # matriz para enviar a
    imshow
34
35 # Máscara para rastrear los puntos con  $|z| \leq 2$ 
36 mask = np.ones(C.shape, dtype=bool)
37
38 for i in range(max_iter):
39     Z[mask] = Z[mask] ** 2 + C[mask] #aplicamos la ecuación  $z = z^2 + c$  para
        la mascara true
40     escaped = np.abs(Z) > 2.0 # True si no pertenecen al conjunto, False si
        sí pertenece
41     newly_escaped = escaped & mask
42     escape_grid[newly_escaped] = i
43     mask &= ~escaped # los puntos que quedan, se niegan los que no estan
44     if not np.any(mask):
45         break
46
47     return escape_grid, (xmin, xmax, ymin, ymax)
48
49
50 def julia(c, xmin=-1.5, xmax=1.5, ymin=-1.5, ymax=1.5, width=1000, height
    =1000, max_iter=200):
51     """Calcula la matriz para un Conjunto de Julia fijado en c.
52     Mantiene c constante y varía las condiciones iniciales z0.
53
54     Parámetros:
55
56     xmin, xmax: rango para el eje horizontal. parte real de c
57     ymin, ymax = rango para el eje vertical. parte imaginaria de c
58     width, height: resolución de la imagen (por defecto 1000  $\times$  1000 =
        106 puntos).
59     max_iter: El límite de iteraciones por punto. Si desíés de 200 pasos  $|z| < 2$  se asume que
        pertenece al conjunto de Mandelbrot.
60
61     Devuelve:
62     escape_grid: Una matriz 2D de enteros de tamaño (height, width). Cada
        celda (i, j)
63     almacena el número de iteraciones en las que ese punto escapó
64     (un valor entre 0$ y 199$), o 200$ (max_iter) si el punto nunca escapó
65     y
66     se quedó atrapado dentro del fractal.
67     (xmin, xmax, ymin, ymax): Una tupla con las coordenadas extremas.
68     Estas se usan para graficar con plt.imshow y graficar los valores físicos
        en lugar del
69     índice de píxeles del arreglo.
70     """
71
72     x = np.linspace(xmin, xmax, width)
```

```
73 y = np.linspace(ymin, ymax, height)
74 Z = x + 1j * y[:, None] # Matriz de condiciones iniciales z0
75 escape_grid = np.full(Z.shape, max_iter, dtype=int)
76
77 mask = np.ones(Z.shape, dtype=bool)
78
79 for i in range(max_iter):
80     Z[mask] = Z[mask] ** 2 + c
81     escaped = np.abs(Z) > 2.0
82     newly_escaped = escaped & mask
83     escape_grid[newly_escaped] = i
84     mask &= ~escaped
85     if not np.any(mask):
86         break
87
88 return escape_grid, (xmin, xmax, ymin, ymax)
89
90 if __name__ == "__main__":
91
92     #c_interesante = -0.7 + 0.27015j
93     #c_interesante = -0.75 + 0.0j
94     #c_interesante = 0.285 + 0.01j
95     c_interesante = -0.8 + 0.156j
96     # Genera matrices numéricas
97     grid_mandelbrot, ext_m = mandelbrot(max_iter=300)
98     grid_julia, ext_j = julia(c=c_interesante, max_iter=300)
99
100     # plot
101     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6), facecolor="black")
102
103     # Mandelbrot Plot
104     ax1.imshow(grid_mandelbrot, extent=ext_m, cmap="twilight_shifted", origin=
        "lower" )
105     ax1.set_title("Conjunto de Mandelbrot", color="white", fontsize=11,
        fontweight="bold")
106     ax1.set_xlabel("Re(c)", color="white")
107     ax1.set_ylabel("Im(c)", color="white")
108     ax1.tick_params(colors="white")
109
110     # Julia Plot
111     ax2.imshow(grid_julia, extent=ext_j, cmap="twilight_shifted", origin="
        lower")
112     ax2.set_title( f"Conjunto de Julia para $c = {c_interesante}$", color="
        white", fontsize=11,fontweight="bold")
113     ax2.set_xlabel("Re($z_0$)", color="white")
114     ax2.set_ylabel("Im($z_0$)", color="white")
115     ax2.tick_params(colors="white")
116
117     plt.tight_layout()
118     plt.savefig('../Clases/Herramientas_Python/julia_manderboot.pdf',
```

```

119     bbox_inches='tight')
    plt.show()

```

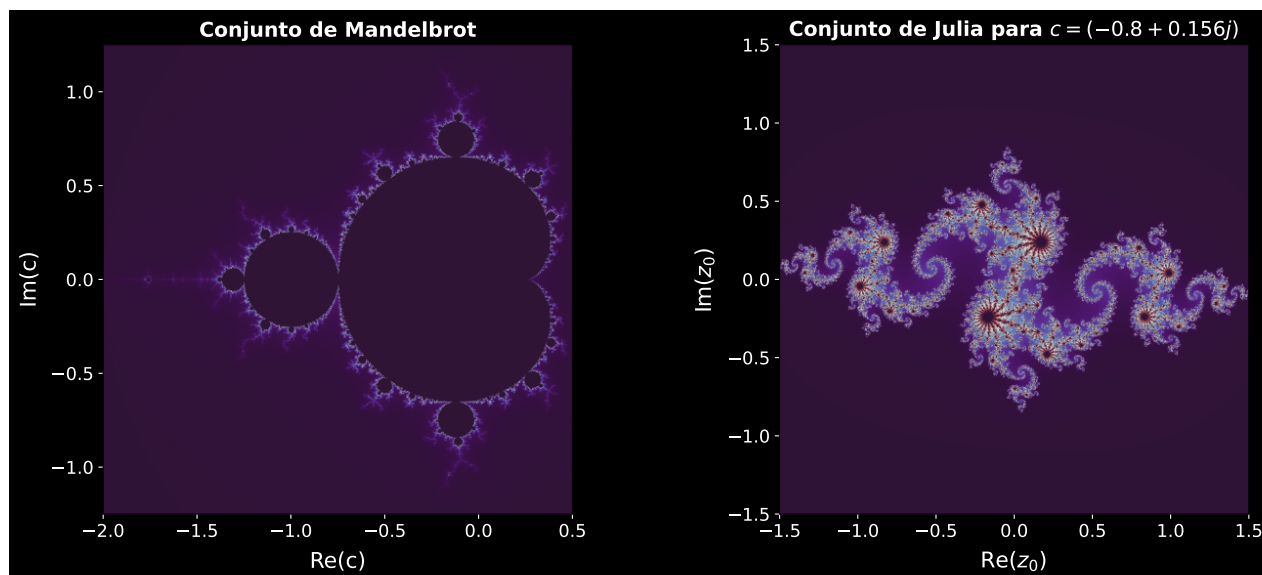


Figura 5: Conjuntos de Julia y de Mandelbrot.

Resumen de las rutinas básicas utilizadas (Fractales e imshow)

◆ Acceda a la documentación oficial `matplotlib.pyplot.imshow` [dando clic aquí](#).

1. `y[:, None]` o `y[:, np.newaxis]`

- **Qué hace:** Inserta un nuevo eje dimensional al vector 1D de tamaño $(N,)$, transformándolo en una matriz columna de dimensiones $(N, 1)$.
- **Uso en broadcasting:** Permite que al operar un vector fila x de tamaño $(M,)$ con $y[:, \text{None}]$, NumPy expanda automáticamente la memoria y construya la cuadrícula bidimensional $C_{ij} = x_j + iy_i$ de dimensiones (N, M) sin requerir bucles anidados lentos.

2. `np.zeros_like(C, dtype=np.complex128)`

- **Qué hace:** Reserva en memoria contigua una matriz de ceros con las dimensiones exactas de C , asignando explícitamente el tipo de dato complejo de doble precisión (128 bits: 64 bits para la parte real y 64 para la imaginaria).
- **Importancia numérica:** Evita truncamientos de coma flotante o excepciones de tipo al calcular potencias y productos con números imaginarios ($1j$).

3. `Z[mask]` y actualización mediante máscaras booleanas

- **Mecánica:** Filtra y evalúa la función iterativa cuadrática $z_{n+1} = z_n^2 + c$ exclusivamente sobre aquellos elementos donde `mask == True` (los puntos cuya magnitud aún no supera la cota de

escape).

- **Eficiencia computacional:** Reduce exponencialmente el costo temporal del bucle; a medida que los puntos exteriores escapan al infinito, la CPU deja de procesarlos y se enfoca únicamente en los bordes críticos del fractal.

4. `newly_escaped = escaped & mask` y `mask &= ~escaped`

- **Qué hace:** El operador bitwise AND (&) aísla los puntos que violaron el radio de escape ($|z| > 2$) en el paso actual i , impidiendo sobrescribir las celdas de píxeles que ya habían escapado en pasos anteriores.
- **Actualización:** `mask &= ~escaped` aplica una negación lógica (~) que apaga a False las posiciones divergentes para excluirlas en las siguientes iteraciones.

5. `ax.imshow(grid, extent, origin, cmap)`

- **Entrada numérica:** Recibe una matriz bidimensional discreta donde cada valor representa la iteración de escape y le asigna un color continuo mediante la paleta `cmap`.
- **Parámetro `origin="lower"`:** Coloca el índice $y_{\min}$ en la parte inferior del eje vertical. Sin este parámetro, Matplotlib usa por defecto la convención matricial (upper), colocando la fila 0 arriba e invirtiendo el plano complejo de cabeza.
- **Parámetro `extent=[xmin, xmax, ymin, ymax]`:** Calibra el sistema de coordenadas de la figura, sustituyendo los índices discretos de píxeles $(0, 1, \dots, N)$ por las dimensiones físicas y matemáticas continuas del espacio complejo.

Problemas:

1. **El conjunto de Mandelbrot**, nombrado en honor al matemático Benoit Mandelbrot, es un *fractal*: un objeto geométrico infinitamente ramificado que contiene estructura dentro de estructura, sin importar cuánto nos acerquemos a mirarlo.

Se genera con una regla muy sencilla sobre números complejos:

$$z_{n+1} = z_n^2 + c, \quad (10)$$

donde z y c son números complejos.

Para un valor fijo de c , comenzamos con $z_0 = 0$ y aplicamos la ecuación una y otra vez. Se dice que el punto c pertenece al conjunto de Mandelbrot si el valor de z no crece indefinidamente hacia el infinito. Matemáticamente se demuestra que si en algún momento la magnitud supera el valor 2 ($|z| > 2$), la secuencia se escapará al infinito con total certeza.

Instrucciones:

- a) Defina una cuadrícula de puntos complejos $c = x + iy$ en la región $-2 \leq x \leq 2$ y $-2 \leq y \leq 2$.

- b) Para cada punto de la cuadrícula, aplique la iteración $z_{n+1} = z_n^2 + c$ comenzando con $z_0 = 0$ hasta un máximo de 100 pasos. Si en algún paso intermedio $|z| > 2$, detenga el cálculo y registre que ese punto escapó.
- c) Dibuje el resultado con un gráfico de densidad (`plt.imshow`). Pinte de negro los puntos que quedaron atrapados dentro del conjunto y de color claro los que escaparon.
- d) **Pista:** Empiece con una cuadrícula pequeña (por ejemplo de 100×100) para probar que su código funcione rápido, y luego aumente la resolución a 500×500 o más.

2. **Los conjuntos de Julia** usan exactamente la misma fórmula que el ejercicio anterior ($z_{n+1} = z_n^2 + c$), pero con los papeles invertidos:

- En lugar de variar c , dejamos una constante compleja c fija para toda la simulación.
- Lo que cambia en cada píxel de la imagen es la condición inicial $z_0 = x + iy$.

El conjunto de Julia reúne a todos los puntos iniciales z_0 que no se escapan a infinito bajo la iteración. Dependiendo del valor de c que elijamos, obtendremos figuras conectadas con patrones intrincados o figuras que se desmoronan en polvo.

Instrucciones:

- a) Elija una constante compleja de prueba, por ejemplo $c = -0.7 + 0.27015i$ o $c = -0.4 + 0.6i$.
- b) Construya una cuadrícula en el plano complejo para $z_0 = x + iy$ en el intervalo $-1.5 \leq x \leq 1.5$ y $-1.2 \leq y \leq 1.2$.
- c) Itere la ecuación hasta 100 veces por cada punto. Guarde en una matriz cuántas iteraciones tardó cada punto en superar la barrera $|z| > 2$.
- d) Muestre la imagen resultante con un mapa de color como "inferno" o "magma". Ensaye cambiando ligeramente el valor de c y observe cómo muta la figura.

3. **El juego del caos y el triángulo de Sierpinski** El juego del caos muestra cómo un proceso totalmente aleatorio puede construir una estructura geométrica perfecta sin necesidad de dibujar líneas directas:

- a) Dibujamos tres vértices fijos en el plano formando un triángulo equilátero:

$$A = (0, 0), \quad B = (1, 0), \quad C = \left(0.5, \frac{\sqrt{3}}{2}\right).$$

- b) Empezamos con un punto inicial cualquiera en el plano, por ejemplo $(0.5, 0.2)$.
- c) Lanzamos un dado de tres caras para elegir al azar uno de los tres vértices.
- d) Marcamos un nuevo punto exactamente en el punto medio entre la posición actual y el vértice seleccionado.
- e) Repetimos el proceso miles de veces, usando siempre el último punto marcado como punto de partida para el siguiente paso.

Instrucciones:

- a) Escriba un script que genere 50 000 puntos siguiendo estas reglas usando el generador de números aleatorios de NumPy.
- b) Descarte los primeros 100 puntos (son el transitorio mientras el caminante entra en la figura).
- c) Grafique todos los puntos acumulados con `plt.scatter` usando puntos muy pequeños (`s=0.1`). Verá formarse el triángulo de Sierpinski, con triángulos vacíos a todas las escalas.

4. **Ramificación recursiva de un árbol** Muchos patrones en la naturaleza, como las ramas de los árboles o los vasos sanguíneos, crecen mediante una regla de división simple que se repite sobre sí misma a escalas cada vez más reducidas.

Podemos modelar un árbol básico con los siguientes pasos:

- a) Trazamos un tronco vertical desde la base con una longitud L .
- b) Al final del tronco, nacen dos ramas hijas: una desviada hacia la izquierda (ángulo $+\theta$) y otra hacia la derecha (ángulo $-\theta$).
- c) Cada rama hija mide un porcentaje de la rama padre (por ejemplo, el 70 %: $L_{\text{nueva}} = 0.7L$).
- d) En la punta de cada rama hija, repetimos la división de forma sucesiva hasta llegar a un límite de ramificaciones (por ejemplo, 8 o 9 niveles).

Instrucciones:

- a) Escriba una función recursiva corta que dibuje este árbol usando segmentos con `plt.plot`.
- b) Pruebe con ángulos de apertura de 20° , 30° y 45° , y note cómo cambia la silueta.
- c) **Opcional:** Añada una variación aleatoria leve en cada bifurcación (en el ángulo y en la longitud) para transformar el patrón geométrico rígido en un árbol de apariencia natural.

5. **El mapa logístico y el caos determinista** En 1976, Robert May popularizó una ecuación elemental para modelar cómo cambia año a año el tamaño de una población de insectos en un cultivo:

$$x_{n+1} = r x_n (1 - x_n). \quad (11)$$

Aquí, x_n es la población normalizada entre 0 (extinción) y 1 (capacidad máxima del entorno), y r es la tasa de crecimiento (entre 0 y 4). El término x_n favorece los nacimientos, mientras que $(1 - x_n)$ simula la limitación por alimento y espacio.

Aunque la fórmula no contiene azar, su comportamiento a largo plazo cambia radicalmente según el valor de r :

- Para valores pequeños de r , la población se estabiliza en un valor fijo.
- Al subir r , la población empieza a oscilar entre dos valores fijos, luego cuatro, ocho...
- Para $r > 3.57$, el sistema entra en un comportamiento caótico: la población nunca se repite y varía de forma impredecible.

Instrucciones:

- a) Tome una secuencia de valores para r entre 2.5 y 4.0 con pasos pequeños (por ejemplo, 1000 valores).
- b) Para cada r , inicie con una población arbitraria (como $x_0 = 0.5$) e itere la ecuación 300 veces sin guardar nada, para permitir que el sistema olvide la condición inicial.
- c) Itere 100 veces adicionales y guarde los valores resultantes de x .
- d) Grafique los puntos (r, x) usando puntos diminutos (`plt.plot(r, x, 'k', alpha=0.25)`). Obtendrá el famoso diagrama de bifurcación con la cascada hacia el caos.

6. **Decaimiento radiactivo con Monte Carlo** El decaimiento radiactivo es un proceso inherentemente probabilístico: es imposible saber cuándo decaerá un núcleo atómico individual, pero sabemos que en cada segundo tiene una probabilidad fija p de transformarse.

Podemos simular este fenómeno átomo por átomo:

En cada segundo, revisamos cada uno de los átomos activos de la muestra y generamos un número aleatorio uniforme entre 0 y 1. Si el número obtenido es menor que p , el átomo decae y desaparece de la cuenta; si no, sobrevive al siguiente segundo.

Instrucciones:

- a) Inicie con una muestra de $N_0 = 5000$ átomos y una probabilidad de decaimiento por paso de $p = 0.03$.
- b) Simule el proceso durante 150 pasos de tiempo. En cada paso, use el generador de números aleatorios para determinar cuántos átomos sobreviven y actualice el total.
- c) Grafique el número de átomos sobrevivientes en función del tiempo.
- d) En la misma gráfica, compare sus puntos de simulación con la curva teórica exponencial $N(t) = N_0 e^{-\lambda t}$, donde $\lambda = p$.
- e) Encuentre visual o numéricamente el tiempo que tarda la muestra en reducirse a la mitad ($N_0/2$) y verifique si coincide con la vida media teórica $t_{1/2} = \frac{\ln(2)}{p}$.